

PathCraft Backend

Steps followed to complete the assignment

1. Environment setup :

- Created the virtual environment with : `python -m venv venv`

2. Application setup :

- I created a rough model of the backend on paper, identified the necessary endpoints, and based on that, determined which models and schemas needed to be created.
- Connected to the database.
- Created all the required schemas and models, and an `__init__.py` file where I imported all the schemas and models.
- Downloaded all the required dependencies and listed them in the `requirements.txt` file.
- Created the endpoints one by one, first in the main file and then moved them to separate router files.
- Wrote the edge cases at the end using AI tools.
- Created a recommendations router to filter and recommend itineraries.
- Populated database with `seed_data.py`.

3 Testing

- Used Thunder Client to test all endpoints and verify edge cases.

4. Deployment

- First, I pushed the code to GitHub, then deployed it on Render and verified that it was working correctly.

Key Decisions Made

1. Chose SQLite as the default database
2. Decided which models and schemas needed to be created.
3. Identified which endpoints would be useful.
4. Choose a modular architecture.

5. Used separate routers for itinerary and recommendation logic to keep API responsibilities well-organized.

Assumptions

1. API consumers/ Frontend will manage user authentication externally.
2. The recommendation engine works on static tags.
3. Each day has its own set of accommodations, transfers and activities.

Challenges Faced & Resolutions

1. In the beginning, I used PostgreSQL as I had experience with it from previous Node.js projects. However, while integrating it with Python, I encountered several compatibility and configuration issues. Due to time constraints, I decided to switch to SQLite for smoother setup and quicker development.
2. Figuring out and designing all the required API endpoints was one of the toughest parts of the assignment and took a lot of time.
3. Unit testing of API responses for deeply nested itinerary structures was a challenge, and used AI tools to assist with writing and validating the test cases.